

1 本节介绍

本节您将学习如何通过控制开发板的GPIO引脚，实现对RGB灯和按键的控制。

1. 如何将GPIO引脚配置为输出模式，通过引脚电平来控制RGB灯的颜色变化。
2. 如何将GPIO引脚配置为输入模式，来检测按键是否按下或松开，去改变RGB灯的亮灭。

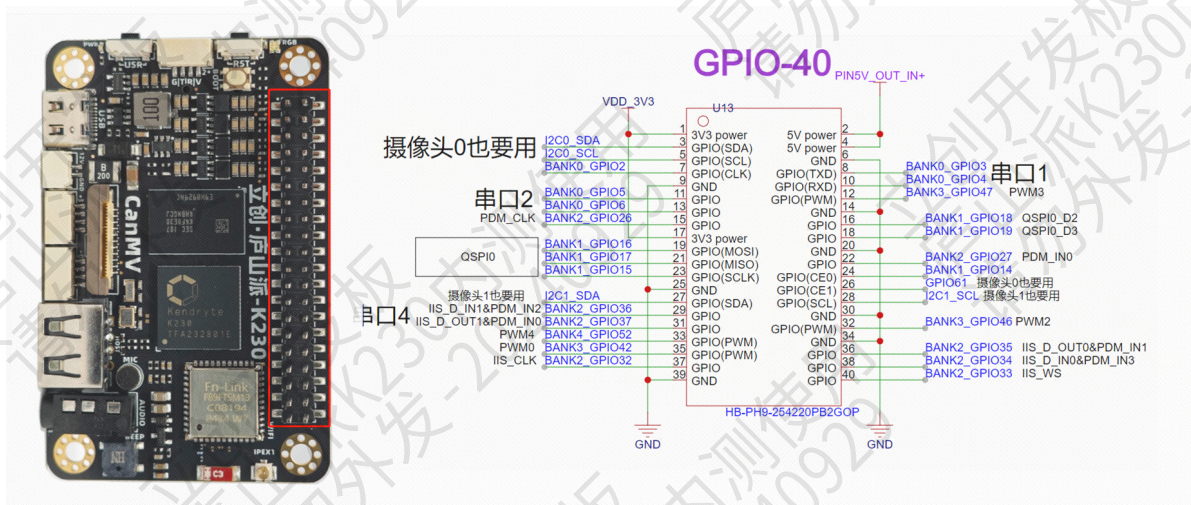
2 名词解释

名词	说明
GPIO	General Purpose Input Output（通用输入/输出）
FPIOA	Field Programmable Input and Output Array(现场可编程 IO 阵列)
iomux	Pin multiplexing(管脚功能选择)
Pin 模块	在MicroPython 中用于控制设备的GPIO 引脚。

1. GPIO就是我们常说的IO引脚，它直接从芯片上引出，可以被设置为输入或输出信号。没有额外功能的GPIO主要用来控制简单的设备，比如点亮LED灯或者读取按键的状态。
2. FPIOA是一种灵活的输入输出阵列，允许芯片对IO功能进行重新配置，类似用STM32上的复用引脚，也就是说不同的引脚可以根据需求重新定义。
3. iomux就是引脚复用了，主要是配置物理PAD(管脚)的功能，由于soc功能多管脚(pads)少，多个功能共享同一个I/O管脚(pads),但是一个pads同一时间只能使用其中一个功能,所以需要IOMUX进行功能选择。IOMUX也叫FPIOA，Pin multiplexing，管脚功能选择等，在K230中，FPIOA就是iomux。
4. 我们使用 MicroPython 语法来控制K230的引脚，machine.Pin 模块用于控制微控制器的 GPIO 引脚。Pin 类提供了访问和控制硬件引脚的功能，主要包括配置引脚模式，读写引脚状态等。

3 开发板引脚介绍

3.1 兼容树莓派40Pin排针引脚



左边是实物图，右边是原理图，该处GPIO还可复用为IIC，SPI，PWM，PDM，IIS等功能，这些IO电平都是3.3V的。

上方的I2C0和I2C1并没有标明实际的IO引脚，而且由于也要驱动摄像头，都已经有了内部上拉电阻了，大家再做外部扩展板或连接外部模块时，就可以不用再加上拉电阻了。这里为了方便大家，不用再去查看原理图，也进行一下GPIO号说明。

I2C0:

- I2C0_SCL : BANK3_GPIO48
- I2C0_SDA : BANK3_GPIO49

I2C1:

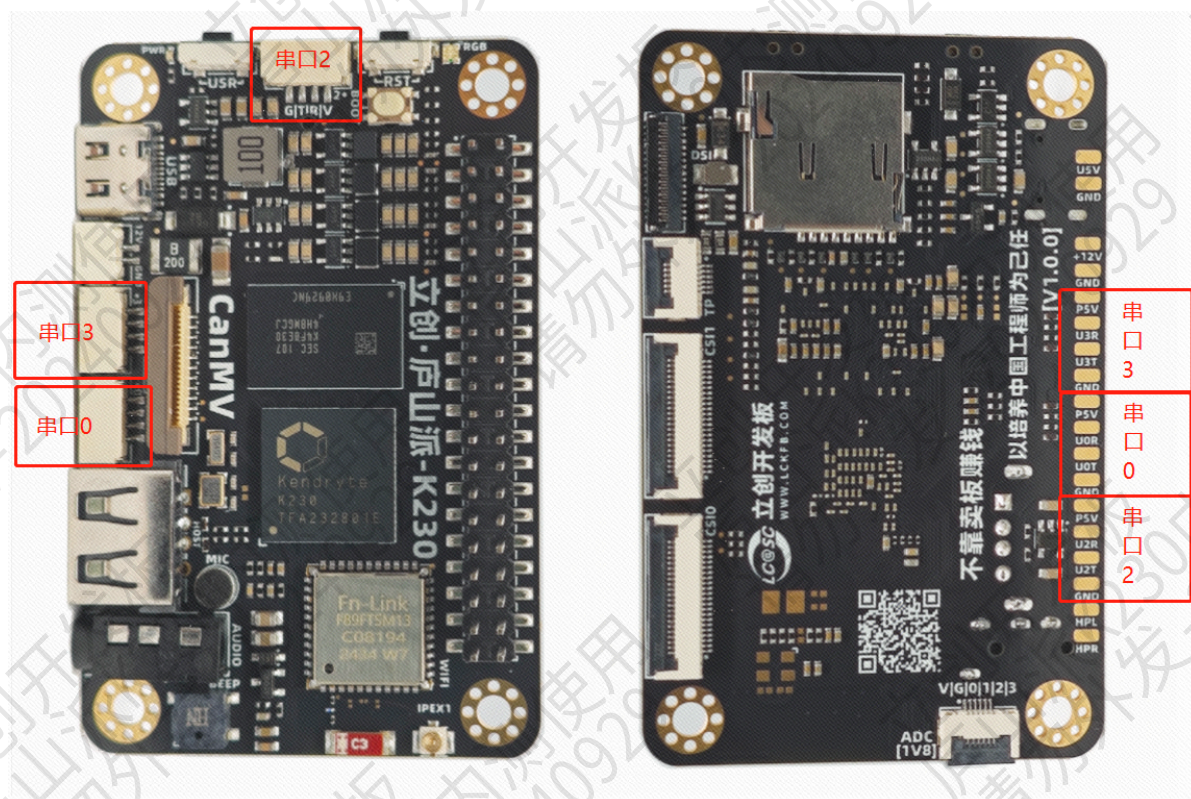
- I2C1_SCL : BANK3_GPIO40
- I2C1_SDA : BANK3_GPIO41

需要注意的是，这里的排针都没有ADC功能，也就是没有模拟功能，这里主要有两个考虑点：

1. 考虑到K230的ADC仅支持最高**1.8V**的输入电压。为了避免用户在使用过程中意外将超过1.8V的电压输入ADC，从而导致芯片损坏，这里并未将ADC接口直接引出到标准的GPIO排针上
2. K230的ADC引脚只能作为模拟输入，不能当做来使用。

结合以上两点本开发板采用了FPC排线座来引出ADC引脚。

3.2 GH1.25座和大焊盘触点引出



左边是立创·庐山派-K230-CanMV开发板的正面图，右边是立创·庐山派-K230-CanMV开发板的背面图，分别用3个GH1.25-4P带锁接口和3组2.54mm间距的大触点焊盘接口对GPIO进行了引出。它们既可以被当做普通GPIO来控制，也可以被复用为串口（UART）来和外部设备进行通讯，其中串口2还可以被复用为IIC2。

串口0:

- UART0_TXD : BANK3_GPIO38
- UART0_RXD : BANK3_GPIO39

串口2:

- UART2_TXD : BANK0_GPIO11
- UART2_RXD : BANK0_GPIO12

串口3:

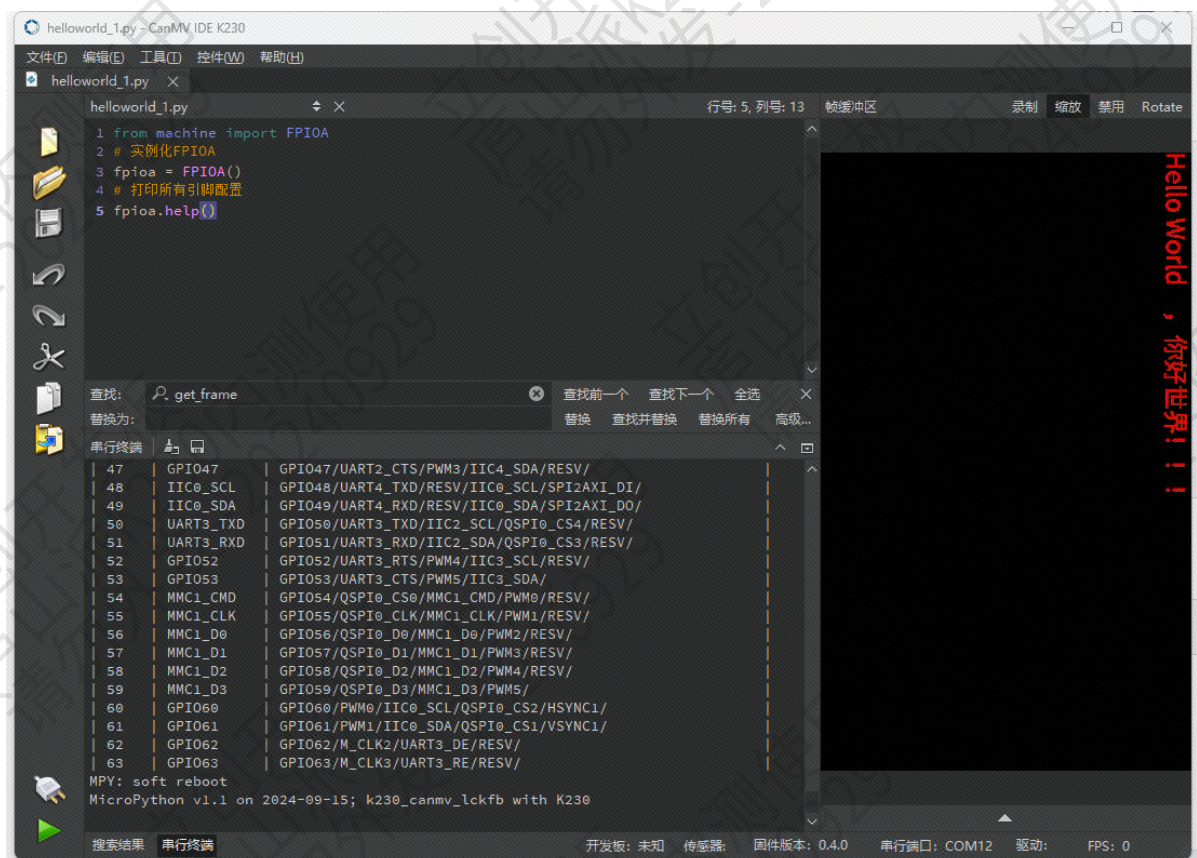
- UART3_TXD : BANK4_GPIO50
- UART3_RXD : BANK4_GPIO51

注意! 串口0默认被系统大核RT-Smart占用, 作为调试串口使用, 所以以上三个串口我们在运行CanMV固件时能用的**只有串口2和串口3**

3.3 各引脚可复用功能

使用以下程序即可打印出各GPIO的复用功能, 把程序复制到CanMV IDE K230里面然后执行就可以从串行终端中得到所有引脚的复用信息了, 为了方便大家查阅, 这里也一并以表格的形式放出来。

```
from machine import FPIOA
# 实例化FPIOA
fpioa = FPIOA()
# 打印所有引脚配置
fpioa.help()
```



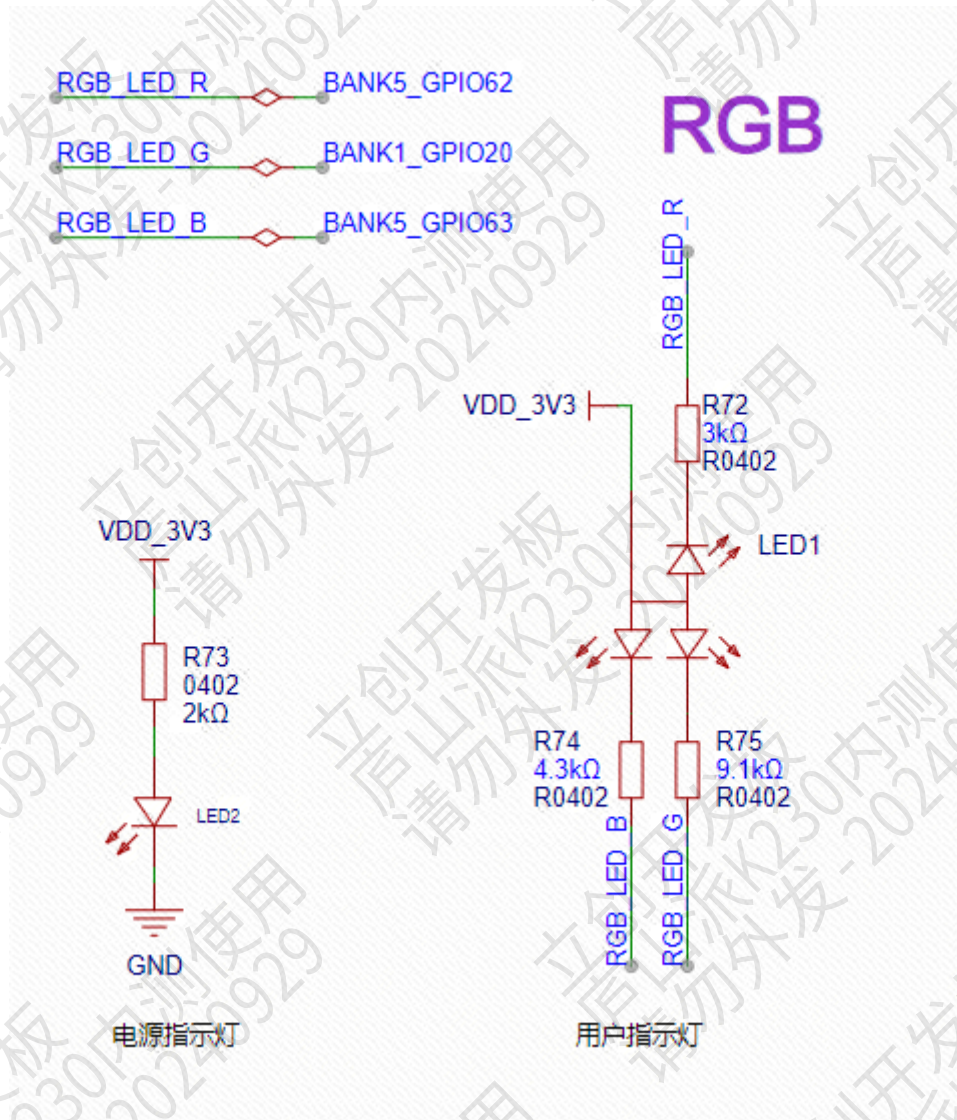
pin	cur func	can be func
0	GPIO0	GPIO0/BOOT0/RESV/
1	GPIO1	GPIO1/BOOT1/RESV/
2	JTAG_TCK	GPIO2/JTAG_TCK/PULSE_CNTR0/RESV/
3	JTAG_TDI	GPIO3/JTAG_TDI/PULSE_CNTR1/UART1_TXD/RESV/
4	JTAG_TDO	GPIO4/JTAG_TDO/PULSE_CNTR2/UART1_RXD/RESV/
5	UART2_TXD	GPIO5/JTAG_TMS/PULSE_CNTR3/UART2_TXD/RESV/
6	UART2_RXD	GPIO6/JTAG_RST/PULSE_CNTR4/UART2_RXD/RESV/
7	PWM2	GPIO7/PWM2/IIC4_SCL/RESV/RESV/
8	PWM3	GPIO8/PWM3/IIC4_SDA/RESV/RESV/
9	PWM4	GPIO9/PWM4/UART1_TXD/IIC1_SCL/RESV/
10	CTRL_IN_3D	GPIO10/CTRL_IN_3D/UART1_RXD/IIC1_SDA/RESV/
11	CTRL_O1_3D	GPIO11/CTRL_O1_3D/UART2_TXD/IIC2_SCL/RESV/
12	CTRL_O2_3D	GPIO12/CTRL_O2_3D/UART2_RXD/IIC2_SDA/RESV/
13	M_CLK1	GPIO13/M_CLK1/RESV/
14	OSPI_CS	GPIO14/OSPI_CS/RESV/QSPI0_CS0/RESV/
15	OSPI_CLK	GPIO15/OSPI_CLK/RESV/QSPI0_CLK/RESV/
16	OSPI_D0	GPIO16/OSPI_D0/QSPI1_CS4/QSPI0_D0/RESV/
17	OSPI_D1	GPIO17/OSPI_D1/QSPI1_CS3/QSPI0_D1/RESV/
18	OSPI_D2	GPIO18/OSPI_D2/QSPI1_CS2/QSPI0_D2/RESV/
19	OSPI_D3	GPIO19/OSPI_D3/QSPI1_CS1/QSPI0_D3/RESV/
20	GPIO20	GPIO20/OSPI_D4/QSPI1_CS0/PULSE_CNTR0/RESV/
21	OSPI_D5	GPIO21/OSPI_D5/QSPI1_CLK/PULSE_CNTR1/RESV/
22	OSPI_D6	GPIO22/OSPI_D6/QSPI1_D0/PULSE_CNTR2/RESV/
23	GPIO23	GPIO23/OSPI_D7/QSPI1_D1/PULSE_CNTR3/RESV/
24	GPIO24	GPIO24/OSPI_DQS/QSPI1_D2/PULSE_CNTR4/RESV/
25	GPIO25	GPIO25/PWM5/QSPI1_D3/PULSE_CNTR5/RESV/
26	PDM_CLK	GPIO26/MMC1_CLK/RESV/PDM_CLK/
27	GPIO27	GPIO27/MMC1_CMD/PULSE_CNTR5/PDM_IN0/RESV/
28	GPIO28	GPIO28/MMC1_D0/UART3_TXD/PDM_IN1/RESV/
29	GPIO29	GPIO29/MMC1_D1/UART3_RXD/CTRL_IN_3D/RESV/

pin	cur func	can be func
30	GPIO30	GPIO30/MMC1_D2/UART3_RTS/CTRL_O1_3D/RESV/
31	GPIO31	GPIO31/MMC1_D3/UART3_CTS/CTRL_O2_3D/RESV/
32	IIS_CLK	GPIO32/IIC0_SCL/IIS_CLK/UART3_TXD/RESV/
33	IIS_WS	GPIO33/IIC0_SDA/IIS_WS/UART3_RXD/RESV/
34	IIS_D_IN0	GPIO34/IIC1_SCL/IIS_D_IN0/PDM_IN3/UART3_RTS/
35	IIS_D_OUT0	GPIO35/IIC1_SDA/IIS_D_OUT0/PDM_IN1/UART3_CTS/
36	GPIO36	GPIO36/IIC3_SCL/IIS_D_IN1/PDM_IN2/UART4_TXD/
37	GPIO37	GPIO37/IIC3_SDA/IIS_D_OUT1/PDM_IN0/UART4_RXD/
38	UART0_TXD	GPIO38/UART0_TXD/RESV/QSPI1_CS0/HSYNC0/
39	UART0_RXD	GPIO39/UART0_RXD/RESV/QSPI1_CLK/VSYNC0/
40	IIC1_SCL	GPIO40/UART1_TXD/IIC1_SCL/QSPI1_D0/RESV/
41	IIC1_SDA	GPIO41/UART1_RXD/IIC1_SDA/QSPI1_D1/RESV/
42	GPIO42	GPIO42/UART1_RTS/PWM0/QSPI1_D2/RESV/
43	GPIO43	GPIO43/UART1_CTS/PWM1/QSPI1_D3/RESV/
44	IIC3_SCL	GPIO44/UART2_TXD/IIC3_SCL/RESV/SPI2AXI_CK/
45	IIC3_SDA	GPIO45/UART2_RXD/IIC3_SDA/RESV/SPI2AXI_CS/
46	IIC4_SCL	GPIO46/UART2_RTS/PWM2/IIC4_SCL/RESV/
47	IIC4_SDA	GPIO47/UART2_CTS/PWM3/IIC4_SDA/RESV/
48	IIC0_SCL	GPIO48/UART4_TXD/RESV/IIC0_SCL/SPI2AXI_DI/
49	IIC0_SDA	GPIO49/UART4_RXD/RESV/IIC0_SDA/SPI2AXI_DO/
50	UART3_TXD	GPIO50/UART3_TXD/IIC2_SCL/QSPI0_CS4/RESV/
51	UART3_RXD	GPIO51/UART3_RXD/IIC2_SDA/QSPI0_CS3/RESV/
52	GPIO52	GPIO52/UART3_RTS/PWM4/IIC3_SCL/RESV/
53	GPIO53	GPIO53/UART3_CTS/PWM5/IIC3_SDA/
54	MMC1_CMD	GPIO54/QSPI0_CS0/MMC1_CMD/PWM0/RESV/
55	MMC1_CLK	GPIO55/QSPI0_CLK/MMC1_CLK/PWM1/RESV/
56	MMC1_D0	GPIO56/QSPI0_D0/MMC1_D0/PWM2/RESV/
57	MMC1_D1	GPIO57/QSPI0_D1/MMC1_D1/PWM3/RESV/
58	MMC1_D2	GPIO58/QSPI0_D2/MMC1_D2/PWM4/RESV/
59	MMC1_D3	GPIO59/QSPI0_D3/MMC1_D3/PWM5/

pin	cur func	can be func
60	GPIO60	GPIO60/PWM0/IIC0_SCL/QSPI0_CS2/HSYNC1/
61	GPIO61	GPIO61/PWM1/IIC0_SDA/QSPI0_CS1/VSYSNC1/
62	M_CLK2	GPIO62/M_CLK2/UART3_DE/RESV/
63	M_CLK3	GPIO63/M_CLK3/UART3_RE/RESV/

4 点亮板载RGB灯

学C语言的第一个程序是Hello world!，那么我们学习嵌入式板子的第一个程序就应该是点亮一个LED灯。那么我们先来看一下开发板上的RGB灯分别连接的是哪些引脚，看一下原理图：



这里的 LED1 就是我们需要控制的用户指示灯，结合右上角的短接符我们可以了解到以下信息：

- 这是一个共阳级的RGB灯，当对应颜色引脚为低电平时对应引脚就会亮。
- 这个RGB灯内部有三个不同颜色的灯珠，**红灯**，**绿灯**，**蓝灯**分别接到了**GPIO62**，**GPIO20**和**GPIO63**上面。

4.1 基础点灯试验

接下来我们实现一个让RGB灯每1秒闪烁一次的程序：

```
from machine import Pin
import time

# 实例化Pin62, Pin20, Pin63为输出, 分别用于控制红、绿、蓝三个LED灯
LED_R = Pin(62, Pin.OUT, pull=Pin.PULL_NONE, drive=7) # 红灯
LED_G = Pin(20, Pin.OUT, pull=Pin.PULL_NONE, drive=7) # 绿灯
LED_B = Pin(63, Pin.OUT, pull=Pin.PULL_NONE, drive=7) # 蓝灯

# 板载RGB灯是共阳结构, 设置引脚为高电平时关闭灯, 低电平时点亮灯
# 初始化时先关闭所有LED灯
LED_R.high() # 关闭红灯
LED_G.high() # 关闭绿灯
LED_B.high() # 关闭蓝灯

# 基础点灯试验: 选择一个LED灯并让其闪烁
# 默认选择红色LED灯, 后续可以通过变量改变需要控制的灯
LED = LED_R # 当前控制的LED为红色LED

while True:
    LED.low() # 点亮当前选择的LED
    time.sleep(0.5) # 等待0.5秒
    LED.high() # 熄灭当前选择的LED
    time.sleep(0.5) # 等待0.5秒
```

在上述程序中, 首先从 `machine` 导入了 `Pin` 模块, 用来控制K230的GPIO引脚, 也倒入了 `time` 模块, 来调用时间延迟函数。

然后实例化了三个GPIO引脚, 分别用于控制RGB灯的红、绿和蓝三种颜色的LED灯, 都指定了对应的引脚号, 输出模式, 不使用上下拉, 输出能力。

💡 构造函数

```
pin = Pin(index, mode, pull=Pin.PULL_NONE, drive=7)
```

【参数】

- index: 引脚号, 取值:[0,63]
- mode: 输入或输出模式
- pull: 上下拉配置, 可选参数, 默认PULL_NONE
- drive: 驱动能力配置, 可选参数, 默认7

接下来初始化了LED灯的状态, 通过让每个灯的控制引脚变为高电平来熄灭所有LED灯。

然后定义了一个变量 `LED`, 初始设置为控制红色LED。这个小细节在后面可以轻松改变 `LED` 变量为其他颜色的灯 (比如如 `LED_G` 或 `LED_B`), 就可以控制不同的RGB颜色来闪烁了。

最后就是一段无限循环的代码, 来控制 `LED` 变量所选RGB颜色的闪烁效果。首先用 `LED.low()` 方法点亮, 然后使用 `time.sleep(0.5)` 延时0.5秒, 再通过 `LED.high()` 熄灭LED, 再次等待0.5秒后继续循环。

4.2 点亮7种不同颜色的RGB灯

接下来我们实现一个让RGB灯组合成七种不同的颜色然后每1秒闪烁一次的程序：

```
from machine import Pin
import time

# 实例化Pin62, Pin20, Pin63为输出, 分别控制红、绿、蓝灯
LED_R = Pin(62, Pin.OUT, pull=Pin.PULL_NONE, drive=7)
LED_G = Pin(20, Pin.OUT, pull=Pin.PULL_NONE, drive=7)
LED_B = Pin(63, Pin.OUT, pull=Pin.PULL_NONE, drive=7)

def set_color(r, g, b):
    """设置RGB灯的颜色, 使用Pin.high()和Pin.low()控制"""
    if r == 0:
        LED_R.low() # 红灯亮
    else:
        LED_R.high() # 红灯灭

    if g == 0:
        LED_G.low() # 绿灯亮
    else:
        LED_G.high() # 绿灯灭

    if b == 0:
        LED_B.low() # 蓝灯亮
    else:
        LED_B.high() # 蓝灯灭

def blink_color(r, g, b, delay):
    """设置颜色并让灯亮一段时间后熄灭"""
    set_color(r, g, b) # 设置颜色
    time.sleep(delay) # 保持该颜色一段时间
    set_color(1, 1, 1) # 熄灭所有灯 (共阳: 1为熄灭)
    time.sleep(delay) # 熄灭后等待一段时间

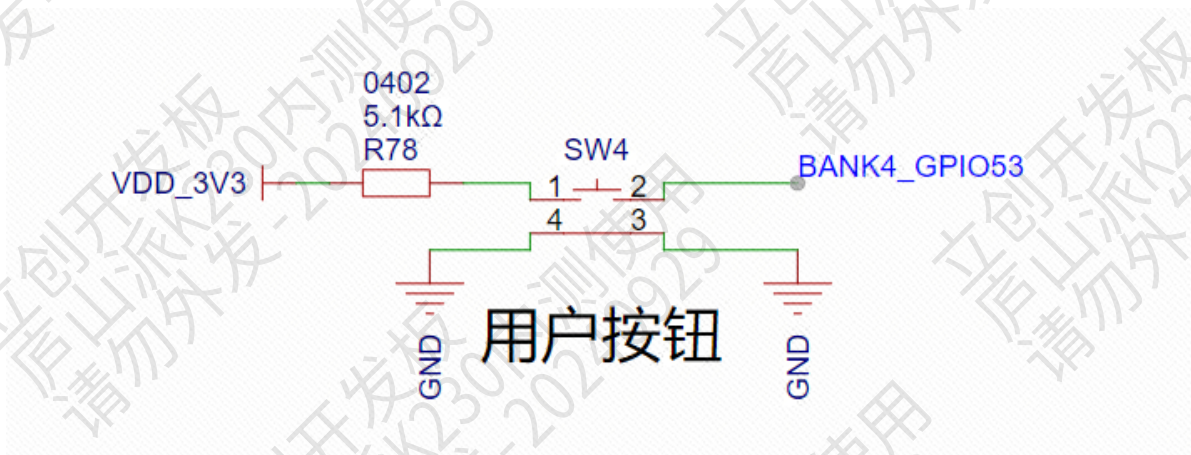
while True:
    # 红色
    blink_color(0, 1, 1, 0.5)
    # 绿色
    blink_color(1, 0, 1, 0.5)
    # 蓝色
    blink_color(1, 1, 0, 0.5)
    # 黄色 (红+绿)
    blink_color(0, 0, 1, 0.5)
    # 紫色 (红+蓝)
    blink_color(0, 1, 0, 0.5)
    # 青色 (绿+蓝)
    blink_color(1, 0, 0, 0.5)
    # 白色 (红+绿+蓝)
    blink_color(0, 0, 0, 0.5)
```

前面和基础点灯试验重复的地方就不再赘述，不同的是这里实现了一个设置颜色的函数 `set_color(r, g, b)`，它用于设置 RGB 灯内部的三个LED的颜色。接受三个参数 `r`、`g` 和 `b`，分别代表红、绿、蓝灯的状态（0 为亮，1 为灭）。函数内部通过判断每个参数的值来控制对应 LED 灯的状态。使用 `Pin.low()` 让对应的灯亮，用 `Pin.high()` 使对应的灯灭，实现了三种颜色的灵活配置。

然后定义了一个让对应RGB颜色闪烁的函数 `blink_color(r, g, b, delay)`，它先设置所需的颜色，然后让 LED 灯保持亮起一段设定时间（通过 `delay` 参数调整）。然后，调用 `set_color` 函数将所有灯熄灭（因为我们庐山派板子上的RGB灯为共阳结构，熄灭时三个灯都设置为高电平），防止影响下一个颜色，再在熄灭状态下等待一段时间。

接下来就到无限循环了，依次调用 `blink_color` 函数，使板载RGB灯以不同的颜色闪烁。

5 使用用户按键



TODO，等待后续完善，需要再写基础些。

5.1 按键控制板载RGB灯亮

当用户按下用户按键时关闭对应的LED灯

```
from machine import Pin
import time

# 实例化Pin62, Pin20, Pin63为输出，分别用于控制红、绿、蓝三个LED灯
LED_R = Pin(62, Pin.OUT, pull=Pin.PULL_NONE, drive=7) # 红灯
LED_G = Pin(20, Pin.OUT, pull=Pin.PULL_NONE, drive=7) # 绿灯
LED_B = Pin(63, Pin.OUT, pull=Pin.PULL_NONE, drive=7) # 蓝灯

# 按键引脚为53，按下时高电平，设置为输入模式
button = Pin(53, Pin.IN, Pin.PULL_DOWN) # 使用下拉电阻

# 板载RGB灯是共阳结构，设置引脚为高电平时关闭灯，低电平时点亮灯
# 初始化时先关闭所有LED灯
LED_R.high() # 关闭红灯
LED_G.high() # 关闭绿灯
LED_B.high() # 关闭蓝灯

# 基础点灯试验：选择一个LED灯并让其闪烁
# 默认选择红色LED灯，后续可以通过变量改变需要控制的灯
LED = LED_R # 当前控制的LED为红色LED
```

```

while True:
    if button.value() == 1:
        LED.high()
    else:
        LED.low()

```

5.2 用按键切换RGB灯状态

```

from machine import Pin
import time

# 实例化Pin62, Pin20, Pin63为输出, 分别控制红、绿、蓝三个LED灯
LED_R = Pin(62, Pin.OUT, pull=Pin.PULL_NONE, drive=7) # 红灯
LED_G = Pin(20, Pin.OUT, pull=Pin.PULL_NONE, drive=7) # 绿灯
LED_B = Pin(63, Pin.OUT, pull=Pin.PULL_NONE, drive=7) # 蓝灯

# 按键引脚为53, 按下时高电平, 设置为输入模式
button = Pin(53, Pin.IN, Pin.PULL_DOWN) # 使用下拉电阻

# 初始选择控制红灯
LED = LED_R # 默认控制红灯

# 初始化时关闭所有LED灯 (共阳: 高电平为灭灯)
LED_R.high()
LED_G.high()
LED_B.high()

# 消抖时间设置为20毫秒
debounce_delay = 20 # 毫秒
last_press_time = 0 # 上次按键按下的时间, 单位为毫秒

# 记录LED当前状态, True表示亮, False表示灭
led_on = False

# 记录按键状态, 用于检测按下和松开的状态变化
button_last_state = 0 # 上次按键状态

# 主循环
while True:
    button_state = button.value() # 获取当前按键状态
    current_time = time.ticks_ms() # 获取当前时间 (单位: 毫秒)

    # 检测按键从未按下(0)到按下(1)的变化 (上升沿)
    if button_state == 1 and button_last_state == 0:
        # 检查按键是否在消抖时间外
        if current_time - last_press_time > debounce_delay:
            # 切换LED的状态
            if led_on:
                LED.high() # 熄灭LED
            else:
                LED.low() # 点亮LED

            led_on = not led_on # 反转LED状态
            last_press_time = current_time # 更新按键按下时间

```

更新上次按键状态

button_last_state = button_state